

I2C Communication algorithms

For EEPROM Slave 24C01

(Referring Data Sheet for AT24CXXA
family)

Features

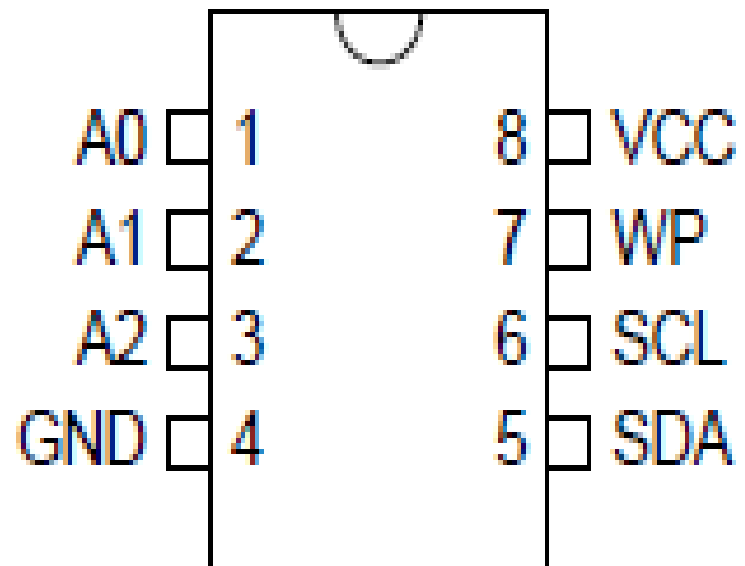
- **Standard-Voltage Operation**
 - **5.0 (VCC = 4.5V to 5.5V)**
- **Internally Organized 128 x 8 (1K)**
- **2-Wire Serial Interface**
- **Bidirectional Data Transfer Protocol**
- **400 kHz (5V)**
- **Write Protect Pin for Hardware Data Protection**
- **8-Byte Page (1K)**
- **Self-Timed Write Cycle (10 ms max)**

Pin diagram

Pin Configurations

Pin Name	Function
A0 to A2	Address Inputs
SDA	Serial Data
SCL	Serial Clock Input
WP	Write Protect
NC	No Connect

8-Pin PDIP



Write Protect Pin Function

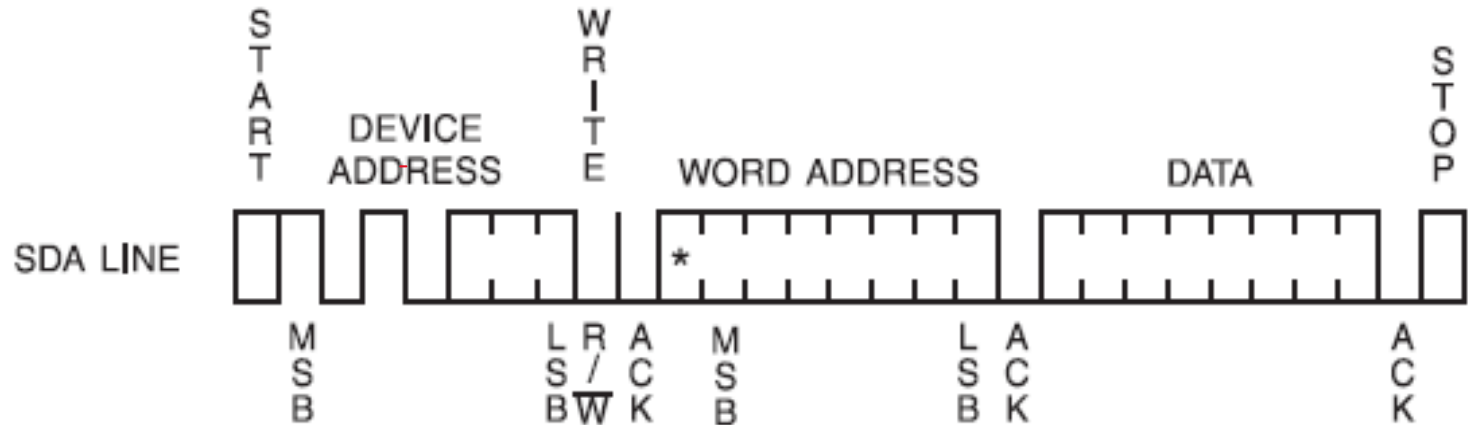
WP Pin Status	Part of the Array Protected				
	24C01A	24C02	24C04	24C08	24C16
At V_{CC}	Full (1K) Array	Full (2K) Array	Full (4K) Array	Normal Read/Write Operation	Upper Half (8K) Array
At GND	Normal Read/Write Operations				

Device Address

1K/2K	1	0	1	0	A_2	A_1	A_0	R/W
	MSD				LSB			
4K	1	0	1	0	A_2	A_1	P0	R/W
8K	1	0	1	0	A_2	P1	P0	R/W
16K	1	0	1	0	P2	P1	P0	R/W

I2C Communication Algorithms

Byte Write

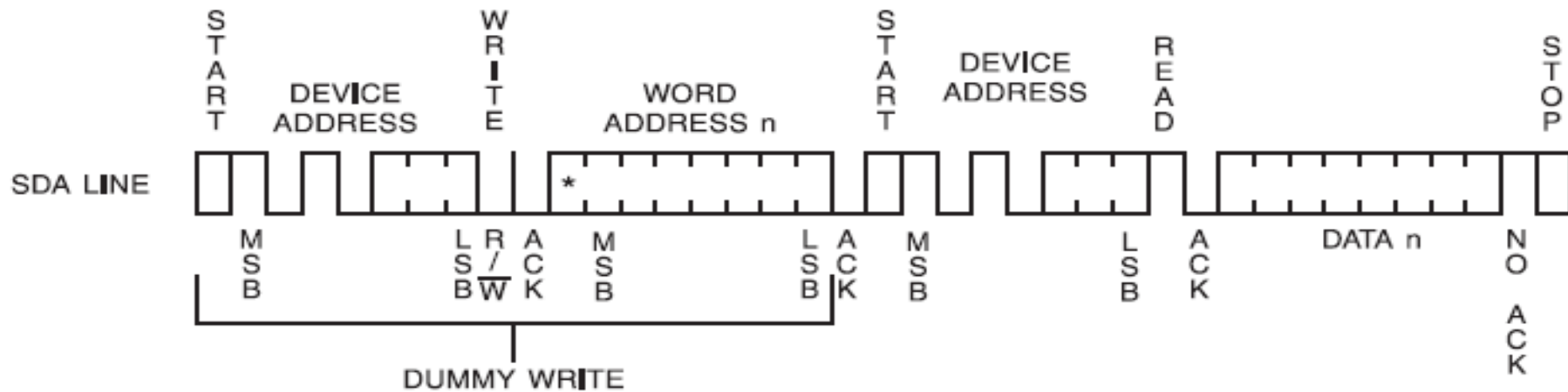


Byte Write (I2C EEPROM 24c01 Communication Algorithm)

- This communication algorithm is used for writing into any random location of the eeprom slave, by each time passing the address of the memory location intended for writing a byte of data into.
- 1) With I2C Bus in idle state, Master issues start condition to begin this communication algorithm
- 2) Master issues 7-bit slave address, with write option
- 3) Master takes acknowledgement from slave
- 4) Master issues byte address of buffer in slave for writing data
- 5) Master takes acknowledgement from slave
- 6) Master issues data(byte) to be written into that address location
- 7) Master takes acknowledgement from slave
- 8) Master issues stop condition to end this communication algorithm
- 9) Master waits for a delay of $\leq 10\text{ms}$ for self-timed write cycle
- Conclusion: min 8 / max 10 i2c bus events make up the above communication algorithm

```
/* file i2c_eeprom.c , Dependencies: LPC21xx.h,types.h,i2c.h,delay.h */  
void i2c_eeprom_write(u8 slaveAddr,u8 wBuffAddr,u8 dat)  
{  
    //Issue Start Condition to begin this comm session  
    i2c_start();  
    //Issue 7bit Slave Address Along With Write Option  
    i2c_write(slaveAddr<<1);  
    //Issue Buffer Address for Writing  
    i2c_write(wBuffAddr); //wBuffAddr  
    //Issue Data for That Write Buffer  
    i2c_write(dat);  
    //Issue Stop Condition to end this comm session  
    i2c_stop();  
    //Delay Period between Consecutive Writes  
    delay_ms(10);  
}
```

Random Read



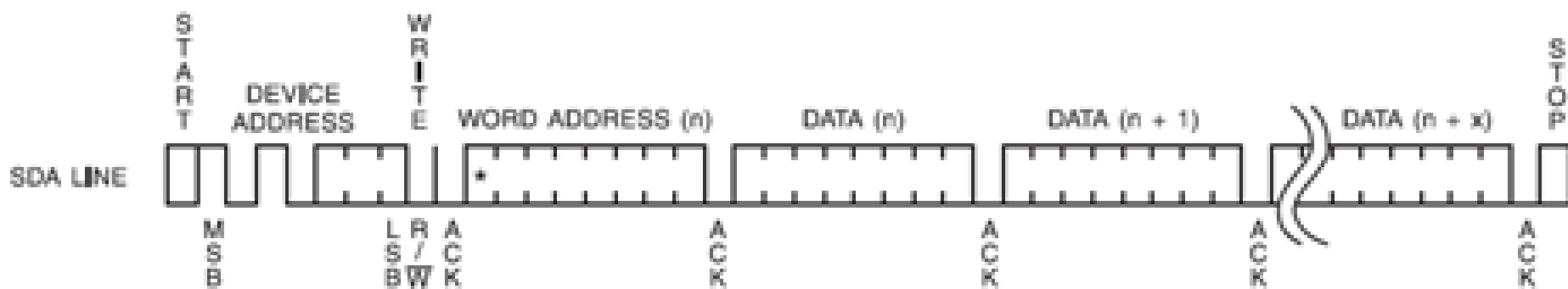
(* = DON'T CARE bit for 1K)

- This Communication algorithm is used for reading a byte from any random location of the eeprom slave by passing each time the address location whose data byte is to be read.

Random Read (I2C EEPROM 24C01 Communication Algorithm)

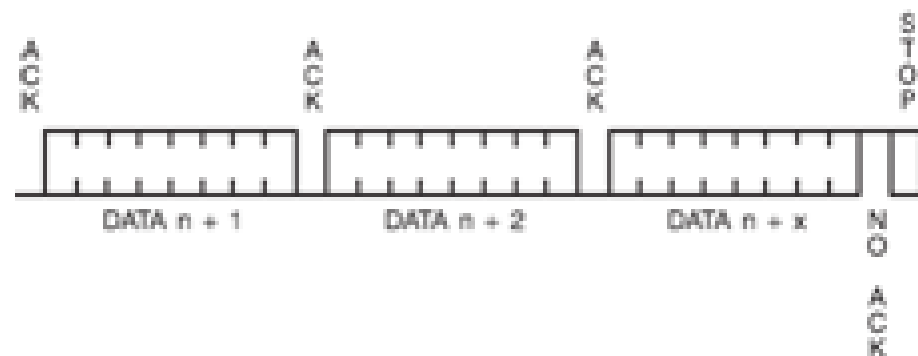
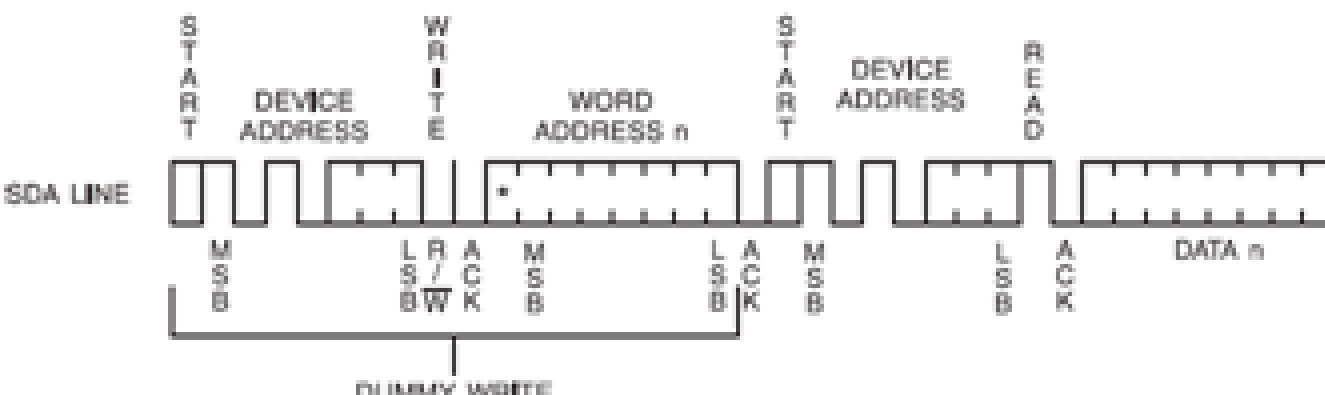
- 1)With I2C Bus in idle State, Master issue start condition to begin this communication algorithm
- 2)Master issues 7-bit slave address with write option
- 3)Master takes acknowledgement from slave
- 4)Master issues byte address of buffer in slave to be read
- 5)Master takes acknowledgement from slave
- 6)Master issues Repeated Start Condition
- 7)Master issues 7-bit slave address with read option
- 8)Master takes acknowledgement from slave
- 9)Master read data(byte) from slave
- 10)Master issues Not Acknowledgment (NACK) condition to inform slave of no further reads
- 11)Master issues stop condition to end this communication session.
- Conclusion : min 11 i2c /max 13 bus events make up this communication algorithm.

```
u8 i2c_eeprom_read(u8 slaveAddr,u8 rBuffAddr)
{
    u8 dat;
    //Issue Start Condition
    i2c_start();
    //Issue Slave Address + Write Option
    i2c_write(slaveAddr<<1);
    //Issue Address of Buffer to be Read
    i2c_write(rBuffAddr);
    //Issue re-start condition
    i2c_restart();
    //Issue Slave Address + Read Option
    i2c_write(slaveAddr<<1|1);
    //Get Not of Ack After Reading Byte
    dat=i2c_nack();
    //Issue stop condition
    i2c_stop();
    //return read byte
    return dat;
}
```



(* = DON'T CARE bit for 1K)

Sequential Read



```
void i2c_eeprom_page_write(u8 slaveAddr,
                           u8 wBuffStartAddr,
                           u8 *p,
                           u8 nBytes)
{
    u8 i;
    i2c_start();
    i2c_write(slaveAddr<<1); //slaveAddr + w
    i2c_write(wBuffStartAddr); //wBuffStartAddr
    for(i=0;i<nBytes;i++)
    {
        i2c_write(p[i]);      //wBuffAddr
    }
    i2c_stop();
    delay_ms(10);
}
```

```
void i2c_eeprom_seq_read(u8 slaveAddr,  
                        u8 rBuffStartAddr,  
                        u8 *p,  
                        u8 nBytes)  
{  
    u8 i;  
    i2c_start();  
    i2c_write(slaveAddr<<1); //slaveAddr + w  
    i2c_write(rBuffStartAddr); //rBuffAddr  
    i2c_restart();  
    i2c_write(slaveAddr<<1 | 1); //slaveAddr + r  
    for(i=0;i<nBytes-1;i++)  
    {  
        p[i]=i2c_masterack();  
    }  
    p[i]=i2c_nack();  
    i2c_stop();  
}
```

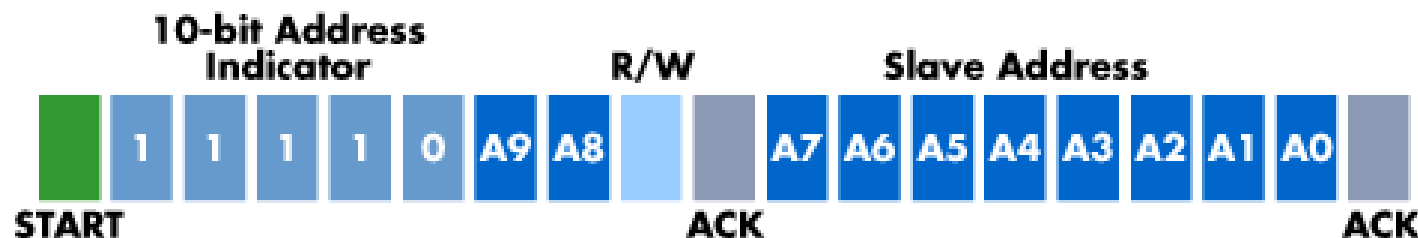
I²C Address, 7-bit and 10-bit formats

- The 1st byte after START determines the Slave to be addressed
- Some exceptions to the rule:
 - “General Call” address: all devices are addressed : 0000 000 + R/W = 0
 - 10-bit slave addressing : 1111 0XX + R/W = X

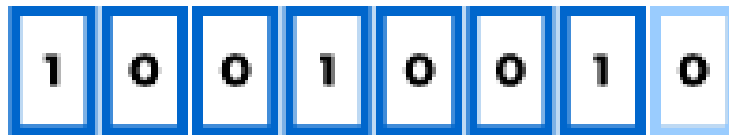
• 7-bit addressing



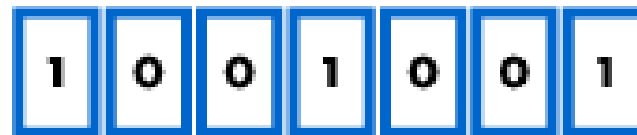
• 10-bit addressing



Write Address: 0x92



Read Address: 0x93



Slave Address: 0x49

Reserved Addresses

000 0XXX

0x00 to 0x07

Valid Address Range

0x08 to 0x77

Reserved Addresses

111 1XXX

0x78 to 0x7f